

CMPT 431 - Project

Christopher M. Peer (Kit)

December 5th, 2024

1 Introduction

This project was created to compare the performance of thread parallel and distributed versions of the LocalPower Iteration Singular Value Decomposition algorithm to a standard serial Jacobi Singular Value Decomposition algorithm. The Local Power Iteration algorithm was invented by Li et al, it is outlined in the paper [4]. A robust and fast method for extracting Singular Vectors from a matrix is useful for many methods in machine learning and data analysis, and any speedups from parallelizing this process would be a valuable pursuit in computation.

LocalPower Iteration is an algorithm designed to find the k^{th} largest Singular Vectors of an input matrix. LocalPower iteration parallelizes the task of Singular Value Decomposition by separating the matrix to be decomposed into equal sized sets of rows. Each process/thread receives a set of matrix rows, then iteratively orthogonalises and transforms a set of output vectors into a set of Singular Vectors (Singularization) respective to the set of rows the process received from the matrix. Every p iterations, the threads/processes collectively sum their output vectors; each thread/process's output vectors make a component of the Singular Vectors, the output vector sum makes the set of Singular Vectors of the whole matrix. Each thread/process then receives a copy of the summed output vectors and continues to iteratively "Singularize" them for p iterations or until the program ends.

The computation time and error of the algorithm depends heavily on p the number of iterations between communication stages:

- With more iterations (larger p) between communication stages, there is less communication overhead and more "useful" computation, decreasing overhead
- With fewer iterations (smaller p) between communication stages, each process/thread generates it's set of Singular Vectors independently, creating more error.
- With more concurrent processing (threads/processes), the input matrix is split into smaller fragments, creating more specialised Singular Vectors in each thread / process, decreasing accuracy, but speeding up the computation time.
- With less concurrent processing (threads/processes), the input matrix is split into larger fragments, creating more generalised Singular Vectors in each thread / process, increasing accuracy, but slowing down computation time.

The intent behind designing the algorithm this way is to balance the tradeoff between Accuracy and Communication overhead by varying the number of iterations between communication stages.

2 Experimental Setup

A serial version of LocalPower SVD is possible, but it is trivial since a serial program has no need to vary the number of iterations between communication stages because it doesn't need to communicate between threads/processes. A robust and fast alternative algorithm for serial computation is the JacobiSVD algorithm provided the Eigen Library [2], which provides a realistic standard in terms of speed and accuracy for a serial program. Each Singular Value Decomposition calculation was done on the coverytype dataset [1], a well conditioned dataset documenting tree coverage.

The LocalPower iteration algorithm was adapted for both distributed MPI and parallel threads versions. For each experiment, the algorithm was run for 16 total iterations to find the top 8 Singular Vectors of the input matrix, with variation on the number of iterations between communication stages, and variation on the number of concurrent processes/threads. With each combination of variables the experiment was run three times: total runtime, cumulative runtime of non-communication iterations, and error were recorded. Error and runtimes were averaged over all three runs.

3 Computation Time

The total computation time of Local Power iteration algorithm in these experiments is dependent on the number of iterations between communication stages (power iterations: p) and the number of threads/processes concurrently processing (nThreads: n). It is important to note that during inter-thread/process communication stages, some useful computation takes place, but it is integral to the communication stage. Calculating the useful computation time does not take this into account and only calculate the iterations where no communication takes place. Although useful work is done during communication stages, the majority of time taken is up by communicating, so the additional useful time is not added.

3.1 Serial Computation

Jacobi Singular Value Decomposition. This algorithm was run as a baseline to compare against the LocalPower algorithm. This algorithm was only implemented and run as serial a serial alternative to LocalPower as no direct serial version for LocalPower exists. This experiment was run using the JacobiSVD function provided by the Eigen library [2]. The average runtime was:

```
Rank: 8
Total time:28.594
Error: 1.33759e-06
```

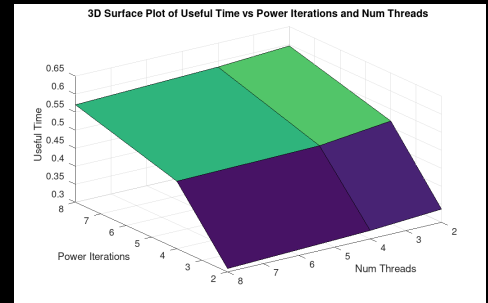
3.2 Parallel Computation

The thread parallel algorithm of LocalPower was implemented using threads.cpp [3] and Eigen [2]. The variables in each experiment were: the number of iterations between communication stages (Power Iterations: p) and the number of concurrent processes (nThreads: n). Each variable was iterated over the range $p, n \in [2, 4, 8]$. For each combination of variables, the experiment was repeated 3 times and the results were averaged. The recorded metrics were:

- The total computation time, from the moment the threads receive input, to the moment they return the Singular Vectors.
- The cumulative computation time for all non-communication computations per each thread (All computations per thread that don't require communicating with other threads)

The results are as follows:

p	n	Total Time	Thread Time
8	8	12.496779	7.12853966666667
4	8	12.558154	6.11907116666667
2	8	12.509493	3.86968729166667
8	4	12.15028366666667	7.092319
4	4	12.44515	6.14964508333333
2	4	12.639199	4.08572125
8	2	12.62732866666667	7.53077983333333
4	2	12.59344266666667	6.509494
2	2	12.6201093333333	4.24881166666667

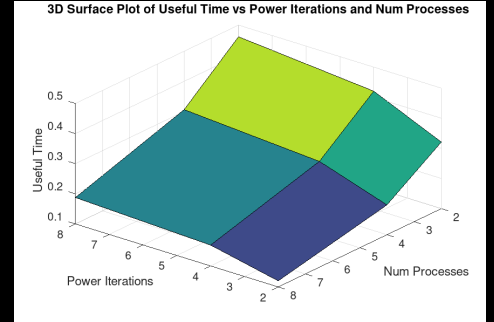


The Ratio of useful time is the cumulative amount of non-communication time per thread, over the total time to compute. $\text{Useful Time} = (\text{Cumulative non-communication time}) / (\text{Total Time})$.

From the graphic and the data above we can see the ratio of useful computation time increases with larger power iterations. With large power iterations the threads spend less time communicating and exchanging information and more time computing. It should be noted that the communication time between threads of a shared memory space is trivial compared to a distributed memory space. Total computation time would not change much with a change in the number of threads because communication between all threads is very quick.

3.3 Distributed Computation

p	n	Total Time	Process Time
8	8	9.06261133333333	1.70064329166667
4	8	8.605155	1.45637983333333
2	8	8.14293933333333	0.979620791666667
8	4	11.23415933333333	3.43791783333333
4	4	10.68674133333333	2.92534958333333
2	4	9.69740633333333	1.93053083333333
8	2	14.92220933333333	6.8825485
4	2	14.05154133333333	5.90270366666667
2	2	12.219839	3.92319266666667



The Ratio of useful time is the cumulative amount of non-communication time per thread, over the total time to compute. $\text{Useful Time} = (\text{Cumulative non-communication time}) / (\text{Total Time})$. From the graphic and the data above we can see both the power iterations and the number of concurrent processes are relevant to the amount of useful computation time. By increasing p the number of iterations between communication stages, fewer communication stages are required, increasing the total amount of non-communication processing time and decreasing the time spent waiting for communications, which improves the Useful Time ratio. Decreasing the number of concurrent processes reduces the total amount of information to be communicated, and the number of sources/destinations to communicate with, making each communication stage less time costly. Even with only 2 processes, this method of computation can still achieve a significant reduction in total computation time compared to the baseline (JacobiSVD, 28.594 seconds), and spends a significant amount of that time computing rather than communicating. With greater numbers of threads and more communication stages the efficiency of this method degrades.

4 Error

Both distributed and parallel version of this algorithm did not perform as well as the serial version as far as error. I suspect this may be due to the implementation rather than the theoretical performance of the algorithm. The Gram-Schmidt orthogonalizer is not as accurate as other methods. This implementation also uses Sign-fixing as an alternative to the Orthogonal Procrustes Transformation (see the paper by Li et Al) [4] which is an $O(n)$ alternative to orthogonalizing on each communication stage, Sign-fixing is fast but lacks accuracy. Improvements to the implementation I provided may be able to improve the error, as well as using more iterations (Larger t) in total before finalising the output matrix.

4.1 Serial Computation

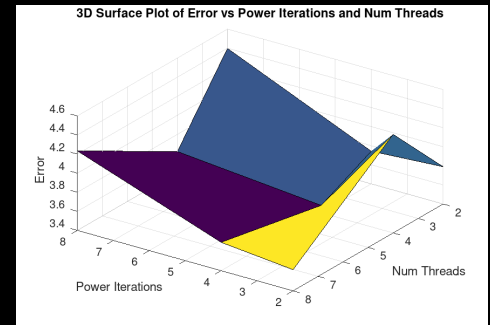
Jacobi Singular Value Decomposition. This algorithm was run as a baseline to compare against the LocalPower algorithm. This algorithm was only implemented and run as serial a serial alternative to LocalPower as no direct serial version for LocalPower exists. This experiment was run using the JacobiSVD function provided by the Eigen library [2]. The average error was:

```
Rank: 8
Total time:28.594
Error: 1.33759e-06
```

4.2 Parallel Computation

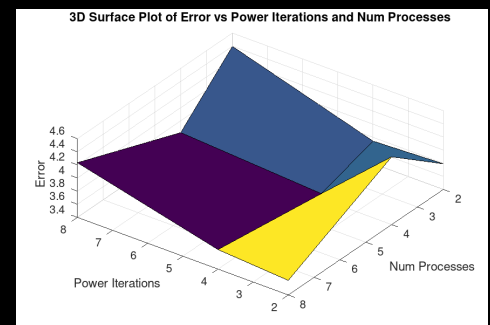
From the graphic and the data provided on the next page, we see error is dependent on both the number of concurrent threads and the number of Power iterations. Larger power iterations (greater p) meant there were fewer communication intervals over the entire computation, meaning each sub matrix was contributing more individually "singularized" vectors. With more concurrent processes (larger n) the original input matrix gets split into smaller pieces, decreasing the granularity of the problem. This had mixed effects on error over the distribution with respect to p .

p	n	Error
8	8	3.580631
4	8	3.17998366666667
2	8	3.81608433333333
8	4	4.53182166666667
4	4	4.66895966666667
2	4	4.28700566666667
8	2	4.201654
4	2	4.30027966666667
2	2	4.42504833333333



4.3 Distributed Computation

p	n	Error
8	8	4.233081
4	8	3.697486
2	8	3.62206366666667
8	4	3.61976
4	4	3.47782166666667
2	4	4.43357633333333
8	2	4.39731933333333
4	2	3.74245
2	2	3.790614



From the graphic and the data provided above, we see error is dependent on both the number of concurrent threads and the number of Power iterations. This is very similar to the results we see from the parallel process, although the distributed process has generally less error. Larger power iterations (greater p) meant there were fewer communication intervals over the entire computation, meaning each sub matrix was contributing more individually "singularized" vectors. With more concurrent processes (larger n) the original input matrix gets split into smaller pieces, decreasing the granularity of the problem. This had mixed effects on error over the distribution with respect to p .

5 Discussion

It is obvious that LocalPower iteration can have a dramatic improvement on speed: both decreasing the granularity of the problem by splitting the input matrix using concurrency, and varying communication efficiency to reduce the amount of communication overhead during computation. I am happy with speed of the computation, but I want to improve the accuracy of this implementation. I suspect the large error in the results are because of a poorly implemented orthogonalizer. In future attempts I would improve this first, and then use larger total iterations to before producing an output. Improving the Sign-fixing method could also help improve the accuracy of the algorithm. In future I would also like to try this algorithm on poorly conditioned input data to see if that affects error, or input data with sparse sections.

References

- [1] Jock Blackard. Covertypes. UCI Machine Learning Repository, 1998. DOI: <https://doi.org/10.24432/C50K5N>.
- [2] Eigen Contributors. Eigen c++ library documentation, 2021. Accessed: 2024-12-05.
- [3] cppreference.com contributors. std::thread - cppreference.com, 2024. Accessed: 2024-12-05.
- [4] Xiang Li, Shusen Wang, Kun Chen, and Zhihua Zhang. Communication-efficient distributed svd via local power iterations, 2021.